Question	Answer
Will the attention score of it change, in case there is a word which is impacting “animal” as well as “it”, and comes after it.	Usually at the encoder steps, masked attention used, meaning only words before affect the attention of the current word
in multi-head attention context, is it multi-self attention head?? each self attention trackes different attention like - animal, pronoun etc ..	Yes. Multi-head attention consists of multiple self-attention heads running in parallel. Each head can learn to focus on different types of relationships, such as pronouns, subject–verb connections, positional information, or semantic concepts like animals, although these roles are learned automatically rather than being explicitly assigned.
How do we calculate how many heads will be required by the model? Will having many heads slow the computation?	The number of attention heads is a hyperparameter chosen by the model designer. More heads can capture different types of relationships, but they also increase computation and memory usage. So, yes, having more heads generally makes the model slower, although the heads are processed in parallel on modern hardware.
What heads refer to in real world, the words to be analyzed?	Not necessarily, a head is an independent attention mechanism, not a word. Each head learns to focus on a different type of relationship in the input (example, pronouns, syntax, nearby words, or semantic meaning). Multiple heads allow the model to analyze the same sentence from different perspectives simultaneously.
so can Oi mean the vector of output from multi head attention?	Yes. o_i is the output vector corresponding to the input token x_i. In multi-head attention, o_i is the final output obtained by combining the outputs of all attention heads
so is Oi, oi1 o12 etc.. represent by seperate neuron? or os ot multiple layers? Seems mul;tiple layers.. since we are caculating an summation?	No, here each attention head produces its own output vector. These vectors are then concatenated and passed through a linear layer to produce the final output o_i. So, the multiple heads operate in parallel within the same layer, not as separate layers.
For QKV xi is input as initially oi is "it" , How xi+1 is dependent on oi ?	X_i+1 is a new word, it is not depends on O_i
do we need to define which all attention head do i need for my exercise like relationships, such as pronouns, subject–verb connections, positional information. Or system by default takes all these heads internally? Also how one filter differed from another one internally, mathematically, can you give an example	Different attention heads are initialized randomly at the beginning and learned during training.
What is the difference between single and multi-head. I think cross - street is similar to it - tired right? Or in cross - street there can be any variable for each that why it's multi-head?	Single-head attention: One attention mechanism looks at the sentence from a single perspective. Multi-head attention: Multiple attention mechanisms look at the same sentence from different perspectives at the same time. Example: The animal didn't cross the street because it was too tired. One head may learn it → animal. Another may learn cross → street. Another may learn animal → tired. So multi-head attention can capture multiple relationships simultaneously, while single-head attention learns only one attention pattern.
so single head don’t much useful in today’s world? or I am missing something?	Yes all the recent LLMs use multihead attention. It helps to capture connection between words from various dimention
can i say single Head is One Detective right ??	Yes correct, it is capturing one of the relation that exists between words
how does Query, key, value - weight matrix calculated in attention heads? How does Wo values calculated ?	All the Q,K,V weight matrices are learned during training process, initialized randomly at the beginning

How does the multi-head attention ensure that information is not duplicated across the heads?	It can be possible two different heads can learn similar connection. But it is highly unlikely that all of them learning same thing as all of them initialized randomly at the beginning. Also learning similar information sometimes helps to pass stronger signal
Will it try to capture relationship of "it" with say "Street" as well?	Yes. Each attention head computes attention between the current token (e.g., "it") and every token in the input, including "Street". If "Street" is relevant in the context, the model can assign it a higher attention weight; otherwise, its attention weight will be low.
self attention -attention within the encoder. cross attention is attention is attention between the enoder-decoder. So multi head is also a self-attention within the encoder?	Yes correct
Ho does this Head work when multiple are being mentioned, eg: "The lion and tiger sit at the top of the food chain, especially the lions which work in groups, so it has an edge. How does the "it" work here?	This is exactly why we use Multi-Head Attention. If we only had one head, it would get confused between the singular "tiger", the plural "lions", and the distance between them. Instead, multiple heads act like a committee. Head 1 might strictly look for singular nouns (lion, tiger). Head 2 might look for the most recently mentioned subject (lions). Head 3 might look for words conceptually related to having an "edge" (groups). Mathematically, the Query vector for "it" Q_{it} takes the dot product with the Keys of all other words. But because each Head has different internal weights, Head 1's dot product might spike on "tiger" (grammar), while Head 2's dot product spikes on "lions" (proximity). By calculating these independently and concatenating the results, the Transformer captures multiple overlapping relationships at the exact same time without them averaging each other out.
Will it try to capture relationship of "it" with say "Street" as well?	Yes in the masked attention everyword influenced by the words before it.
How heads are chosen ?	The number of attention heads is chosen by the model designer as a hyperparameter (e.g., 8, 12, or 16). The heads themselves are not manually assigned specific roles. During training, each head automatically learns to focus on different patterns because it has its own learnable parameters.
How can we decide multiheads should be use din which scenario.How actually it will work	Good Thnking, The short answer is you use it always! Multi-Head Attention isn't an optional feature it is the fundamental building block of all Transformers. Instead of deciding when to use it, you decide how many heads to configure (e.g., 8, 12, or 96) based on how complex your data is and how much compute power you have. More heads = the ability to capture more diverse relationships, but it requires more memory.
Even though it parallelises, but self attention need the preceding context, so isnt is of any consequence, because without the preceding context it is meaningless	The key point is that self-attention does not depend on previously computed outputs. Each token attends directly to the input tokens, not to the output of the previous token. In the encoder, every token can see the entire input sentence, so all outputs can be computed in parallel. In the decoder, although a token can only attend to preceding tokens (due to masking), all masked attention computations for the known input sequence during training are still performed in parallel because the mask already enforces the correct context. During inference, however, tokens must be generated one at a time since future tokens are not yet available.
If all the heads learn the same thing when all of them are initialized randomly at the beginning, how do we solve this?	Different parameters and training dynamics usually lead to specialization, and additional regularization can further reduce duplication.
In the self attention every word in the query will have its attention captured with every other word ? Beacuse in example it and animal only is used just to explain right ?	Yes. In self-attention, every token computes attention with every other token (and itself) in the sequence. The example using only "it" and "animal" is just for illustration. In practice, the model computes attention scores between all pairs of tokens in the input sentence.

What is Add & Norm?	Skip connection & Layer Normalization (Lecture 3)
I am not very clear on Query key and value, can you please explain again, thanks in advance	Query (Q): What am I looking for? Key (K): What information does each book contain? Value (V): The actual content of the book. The model compares the Query with all Keys to find the most relevant tokens, then uses their Values to build the output. Query asks, Key matches, Value provides information.
Why is masked MultiHead Attention not required in encoder?	Because the encoder receives the entire input sentence at once, every token is allowed to attend to every other token. There are no future tokens to hide. Masking is only needed in the decoder to prevent a token from seeing future words during text generation.
how step 4 exactly works summation of softmax and values?	yes
What is Outputs (shifted right)? Can someone give the overall picture of the problem?	Outputs (shifted right): means the decoder is given the correct output sentence, but shifted by one position during training. Example: Target sentence: I love AI <EOS> Input to decoder (shifted right): <BOS> I love AI The model learns: Given <BOS> → predict I Given <BOS> I → predict love Given <BOS> I love → predict AI The decoder always uses previous words to predict the next word, never the future words. This teaches it how to generate text one token at a time.
In Step 4 , why do we need to take weighted sum of output? Is it for one word or across sentence?	The weighted sum is computed for one word at a time. For a token like "it", its attention weights (from the softmax) are used to take a weighted sum of the value vectors of all words in the sentence. This produces a context-aware representation for that specific word.
Why do we mask attention in output embedding and why is it only in past events?	We mask attention so the decoder cannot see future words while predicting the next word. This forces the model to generate text one word at a time, using only the past and current context, just as it would during inference.
For each token or word, how many queries are considered? How do we actually form queries around word?	Each token has one query vector per attention head. The query is not written manually or formed as a natural-language question. It is created by multiplying the token embedding by a learned weight matrix: W_q (Lecture 3). During training, the model learns what information this query should capture.

Unable to understand key purpose of encoder and decoder. i can follow the attention process but unable to relate big picture of encoder and decoder	The Encoder is the Understander. Its only job is to read the input text all at once, figure out the context, and compress that meaning into a dense mathematical format. The Decoder is the Generator. It takes that mathematical meaning and writes the output text one single word at a time. Okay, now you will ask Why do we need both? Because translating or summarizing requires two completely different skills. You need full, bidirectional visibility to understand a sentence (Encoder), but you need strict, step-by-step grammatical rules to generate a new sentence (Decoder). The original Transformer paired them up to get the best of both worlds.
Is cross attention not present in encoder only model or decoder only model? If present, then from where does Q, K and V taken from?	No. Cross-attention is present only in encoder–decoder Transformers. Encoder-only models (like BERT) and decoder-only models (like GPT) use only self-attention. In cross-attention, the queries (Q) come from the decoder, while the keys (K) and values (V) come from the encoder outputs.
All GPT models are based on decoder only architecture. So, how do they understand the context of the input text?	It understands context through a process called the Pre-fill Phase! Before the model generates a single word, it takes your entire prompt and processes it all at once through its Self-Attention layers. Every word in your prompt calculates attention scores with the words that came before it. This creates a dense mathematical representation of your prompt called the KV Cache. That cache is the model's understanding.
For masked attention, why is it j>=i for negative infinity? if j=i, it is the same position right? Should that not be calculable? If j>i, then it is future word and cannot be calculated.	yes you are correct, it was a typo in the picture but it was correct in the table.
How do we decide which architecture should be used between self-attention and cross-attention? I understand that we dont have to use both of it together most of the times.	Self-attention is in Encoder step, cross-attention is used while decoding
As i can understand there are multiple approaches available while we train any model- there are different params that helps engineer decide what to chose. this is getting very confusing- how can we get this understanding clear without any ambiguity specially building such systems in lets say python.	It is totally normal to feel overwhelmed! The secret to removing ambiguity is a top-down approach. Pick the Architecture based on the task: Encoder for understanding (BERT), Decoder for generation (GPT). Don't start from scratch 99% of engineers download a pre-trained model and fine-tune it. Use standard libraries, You don't have to code the math yourself! In Python, we manage all this complexity using libraries like Hugging Face transformers and PyTorch, Its very easy to do plug and play with these withvery little modifications.
In decoder, so does it mean Key and Value vectors come from encoder layer?	Yes in cross-attention K and V comes from encoder layer
is Cross attention is used only in translation? is there any other scenario where cross attention works ?	For all encoder-decoder architecture cross attention used. Lets say summarization tasks where encoder encodes the document and then decoder generates the summary

Does the encoder decoder architecture runs for both the sequences separately ?	es. In an encoder-decoder Transformer: The encoder processes the entire input sequence and creates contextual representations. The decoder processes the output sequence token by token and generates the final output. The two components work on separate sequences, but the decoder can access information from the encoder through cross-attention. For example: Encoder input: "I am a student" Decoder output: "Je suis étudiant" So, the encoder handles the input sequence, while the decoder generates the output sequence.
How does cross attention masking work in a language that writes right to left like Arabic	thats a good question. The masking rule is based on the token order, not the writing direction. For Arabic, the text is tokenized in its natural order, and the decoder still masks future tokens in that sequence. Cross-attention itself is not masked—the decoder can attend to all encoder outputs regardless of whether the language is written left-to-right or right-to-left.
why did cross attention evolve? when self attention technique can actually calculate the weights of every other word in the sentence, what problems from self attention does cross attention solve?	self attention has no mechanism to look outside its own sequence, cross attention is needed because models needed a principled way to read across two separate sequences without merging them.
From self-attention to cross-attention, encoder and decoder are introduced directly in the diagram, but not explained why and how encoder and decoder emerge, can you please explain?	Encoder = understand input, Decoder = generate output. They were introduced to handle input-output sequence tasks such as translation.
so WK and Wv matrix from encoder for cross attention is that right?	yes
why only 8 in multi head? is it architectural decision	yes the number of heads are hyperparameters....
how does decoder know when to stop predicting next word or it is finised?	Wow, didnt expect this question today, youre so right The model actually has no concept of "being finished" Instead, it relies on a special hidden token called the End-of-Sequence (EOS) token like <endoftext>. During training, we put this token at the end of every document. The model learns to predict this token when a thought is grammatically complete. When we run the model, it just loops continuously until the predicted output is the <EOS> token, which triggers our code to stop the loop. I Think of it like saying "Over" on a walkie-talkie.
As part of the cross-attention, the similarities of the word is mapped by the model but how it is taking care of grammer to make the sentence correct?	Cross-attention mainly helps the decoder find the relevant information from the source sentence. Grammar is not handled by cross-attention alone. The decoder's masked self-attention captures the grammatical structure of the sentence generated so far, while the feed-forward layers and the model's learned parameters combine this information with cross-attention. Since the model is trained on millions or billions of grammatically correct sentences, it learns grammar patterns during training and uses them when generating the next token.

if the transformers have masked multi head self attention, and it can't decode the future tokens, then self attention transformer can't predict next word,(for ex email assitance which predict next word). RNN would be a better option for next word prediction. however for translation and textual context understanding systems, transformer based system are good to use.	This is a very logical thought, but there's a mix-up between Training and Inference. We only use the Causal Mask during Training. Because during training, we already have the full sentence, and we have to "mask" or hide the future words so the model doesn't cheat by looking at the answer. If we didn't mask it, it would never learn how to predict! When you use it in the real world (lets say email autocomplete), this is called Inference. In inference, you are typing live. There is no future text to mask because you haven't typed it yet. The Transformer looks at the past words you have typed, and uses what it learned during training to predict the next word. Good that you mentioned RNN, GPT is literally a massive masked self-attention model, and it is vastly superior to an RNN for next-word prediction, Remember last week's session that RNNs suffer from vanishing memory, they forget the start of your email by the time they reach the end.
In real life examples, where do we use self attention, cross attention and both?	self attention used when the sentence needs to understand itself Cross-attention is used when two inputs needs to interact and both are needed when first we first need to understand one input and then interact with the other
ChatGpt like models are Decoder only model, so how do they work without an encoder.	ChatGPT-like models are decoder-only Transformers. They do not separate the input and output into encoder and decoder parts. Instead, the prompt and generated text are treated as one continuous sequence. Example: Question: What is AI? Answer: The model reads the entire prompt using masked self-attention and then predicts the next token: Question: What is AI? Answer: Artificial Then it predicts the next token, and so on. So, instead of using an encoder to understand the input and a decoder to generate output, a decoder-only model uses masked self-attention over the entire context and generates text one token at a time.
can u please explain linear and non linear	Linear attention: Faster and uses less memory for long sequences. Non-linear (standard/softmax) attention: Captures richer relationships between tokens but is slower and uses more memory. In short: Linear attention focuses on efficiency, while non-linear attention focuses on modeling power.

so both are outputs and decoder basically gets input from encoder output and decode it further in encoder-decoder based transformer?	Yes, you have the big picture exactly right! The Encoder's output becomes the "source material" for the Decoder. However, technically speaking, the Decoder receives TWO inputs! It receives the Encoder's output (to know the meaning of the original sentence). + It receives its own partial output (to know what words it has already generated so far). AND, It combines both of these inputs to figure out what the next word should be!
where is the temperature fits in the transformer model ?	it controls the randomness in the generation... will be explained in next lecture
can u explain FFN again and how RElu willl be used	The Feed-Forward Network (FFN) processes each token independently after the attention layer. It consists of two linear layers with an activation function (such as ReLU) in between: FFN(x) = W2(ReLU(W1x+b1))+b2 ReLU sets all negative values to zero and keeps positive values unchanged, introducing non-linearity so the model can learn more complex patterns than a simple linear transformation. (Lecture 3)
how do we get “english” output from the predict vector?	I guess you are refering to the language as english ... The decoder is trained on English text. At each step, it predicts the next English word from the English vocabulary. By repeatedly predicting one English word at a time, the complete English sentence is generated.
Position alone isn't enough right, how do we tell models about the grammar, whether the reference is about the subject or the object	Model learns those things during the training phase. While predicting the next token if it makes mistake, the loss will be high because in the actual data things are different, so from that loss signal it corrects itself
Why position 3 dim1 is negative?	Dim 1 = cos(pos), and pos is in radians. cos(3) ≈ -0.99 because 3 radians ≈ 172°, which falls in the second quadrant (90°–180°), where cosine is negative. That's it — fast dimension, just lands on the negative part of the cosine wave at pos=3.
What is postion 1 and postion 7 can could you give context	Position 1: p = [0.8415, 0.5403, 0.0251, 0.9997, 0.0006] → x = [1.6415, 1.4403, 0.2251, 1.3997, 0.6006] Position 7: p = [0.6570, 0.7539, 0.1749, 0.9846, 0.0044] → x = [1.4570, 1.6539, 0.3749, 1.3846, 0.6044] Context: Fast dims (0,1) change a lot between pos 1→7; slow dim (4) barely moves — same "clock hand" pattern as the slide's pos 3 vs 7 example.
Why should a later position of ‘bank’ alter the word vector more?	It doesn't. The position itself does not make the word vector more important. Positional encoding simply tells the model where the word appears in the sequence. The model then learns during training whether that position is important for the task. For example, "bank" at different positions may have different meanings depending on the surrounding context, not because later positions are inherently more significant.

Why do we need diffferent dimension of the word embeddiings to vary seperatly after positional encoding. What is the use of it. It implies that in word embeddings the intial dimenion have more sensitivity in models?	Vikas, you have good observation, But no, it does not mean the initial dimensions are "more sensitive" or more important for meaning. Instead, think of the varying dimensions like the hands of a Clock. The early dimensions (fast-moving sine waves) are like the "seconds hand" they track very short, local distances between neighboring words. The later dimensions (slow-moving waves) are like the "hours hand" which track long, global distances across a whole paragraph! The word embeddings themselves are randomly initialized and learned, there is no inherent hierarchy to their 512 dimensions.
can you explain ROPE and ALBI??	RoPE: Adds position information to tokens so the model knows the order of words. ALiBi: Encourages the model to pay more attention to nearby words and less to distant words. RoPE encodes position, while ALiBi biases attention based on distance.
rhis ROPE alibi is fundamentally not clear. Prof should explain with some example	ROPE and ALiBi are still not very clear to me. The intuition behind them is difficult to understand from the equations alone. It would be very helpful if the professor could explain them with a simple example showing how positional information is incorporated into attention and why these methods work better than absolute positional embeddings.
Why do we take transpose of the Key vector in the self attention formula? What is the derivation of the formula?	Its just Pure linear algebra! Our Query matrix Q has a shape of [N * d_k] and our Key matrix K is also [N * d_k]. You cannot multiply two matrices of those shapes. By transposing K into [d_k * N] the inner dimensions match. Multiplying [N * d_k] by [d_k * N] perfectly results in an [N * N] matrix Which is exactly the [N * N] attention grid we need to show how every word relates to every other word.
what's BOS ?	Beginning of Sentence
what is mean by decoder only or encoder only model? A model can use both?	it is only using the encoder block or decoder block or both. Prof will explain in coming slides more thoroughly
At which steps in a Transformer does MLP is being used?	MLP applied over the self -attention layer output
Will W_Vocab have all the possible words in the models vocab ?	Yes, W_vocab has one output score for every word (or token) in the model's vocabulary. After multiplying the decoder output by W_vocab, the model produces a score for each vocabulary token, and the token with the highest probability is selected as the prediction.
Everytime it will generate same 50 words?	depends on some more parameters like temperature, num_beans, etc will be discussed in next lecture
why start with BOS and not with something that we want the translation for?	BOS is learned special token, telling the model we are starting generation.
How does we decide max_len? .. why its 50	It is a design choice, nothing magical with 50, it depends on your dataset

what is ligits? Prof just read through it but not explained why to use it and in which case.	I think you mean “logits” ? Logits are the raw unnormalized scores that the neural network spits out right at the very end. For example, if the model is predicting the next word, it might assign a logit of 12.5 to "cat", 3.1 to "dog", and -50.2 to "car". Because these numbers are arbitrary and don't add up to 100, we pass them through a Softmax function to turn them into clean percentages.
what is query in decoder in shown example? BOS + new tokens?	Yes. In the decoder, the queries are computed from the decoder input, which consists of the previously generated tokens (starting with the beginning-of-sequence token). At each step, the decoder uses these tokens to form the query vectors, while the keys and values in cross-attention come from the encoder output.
Please explain Auto regressive decoder steps	An autoregressive decoder generates the output one token at a time. At each step, it uses all previously generated tokens (through masked self-attention) and the encoder output (through cross-attention) to predict the next token. The newly predicted token is then fed back as input to generate the following token, and this process continues until the sentence is complete.
how to prepare the dataset for training the model ? how to evaluate the quality of dataset ? what are the techniques to refine data ?	Woah, this is a big process, its a very tedious one Preparing data for GenAI requires Scraping, Cleaning, Tokenization and Formatting, and each step has its own challenges in between, Scraping is itself a big challenge , you have tons of resources, for Cleaning, if its HTML you have to remove HTML tags, fix bad formatting all of that, formatting could invlove you <BOS>, <EOS> tokens for every fixed length sequences. For Measuring QUality of dataset, Diversity, Are you feeding it too much of one domain (like only legal documents)? Toxicity, Running secondary "classifier" models over your data to flag and measure hate speech or unsafe content. Perplexity/Validation Loss where we traini a tiny, miniature model on a sample of the data to see if it actually learns anything useful.
Now question is..Auto regressive decoder...who gives them the token which they need to work upon	During training, the correct previous tokens (ground truth) are provided to the decoder. During inference, the decoder starts with the beginning token, predicts the first token, feeds that prediction back as input, then predicts the next token, and repeats this process until the sentence is complete.
Is the MLM and CLM related to SGP and CBOW that we learnt 2 weeks ago? For MLM SGP is on and for CLM CBOW?	They are related in idea, but they are not equivalent. Both pairs involve predicting missing or neighboring words, but they are used in different models. MLM is somewhat similar to CBOW because it predicts a masked word using its surrounding context, while CLM is somewhat similar to Skip-Gram because it predicts future tokens from the current context. However, MLM/CLM are Transformer objectives, whereas CBOW/Skip-Gram are Word2Vec training methods.
For Document Intellegigence for Caluse, Terms Do we need lot of sample Contracts/Documents to train? or with Modern Transforment it can Understand and propose clause and find issues	there are already some pre-trained transformers, you can use them. Else if you train from scratch you will need lot of data and will cosume a lot of resources

Can you explain when we use Encoder-Only, Decoder-Only, and both (Encoder-Decoder) — with real-life examples and strong clues for when to use what?	Encoder only models are used when you only need the overall input representation, BERT model is Encoder only, basically for classification task encoder type model used where you get the overall sentence representation and use it to classify the sentence. Decoder only model used when we only need to generate free text, like GPT model, chatbots And we need encoder and decoder model when you need to understand input first and then generate. Like Summarization, translation type of task. T5 is a encoder decoder model
If chatgpt, gemini and all commericial models use transformer which are predominantly text based models ... how are they also good with numerical data and analysing vast quantities of numbers for research and analytics. How exactly do they work with numbers and it's context and meanings. Clearly numbers have different meanings from words.	When you ask ChatGPT or Gemini to analyze a huge spreadsheet, the neural network does not do the math itself! Neural networks hallucinate. Instead, the LLM writes a Python script in the background, runs that script in a secure sandbox, and then reads the exact, perfect output to you. It uses its language skills to write code, and lets the code do the math!
So if I need to translate a document from a different language to a standard language (english), we will use the combination of encoder and decoder right?	yes will be demonstrated in tutorial
is language translation best done in encoder decoder model?	yes
Suppose a dataset having English Bengali language both. which model is best ? Like : The weather is so beautiful today, চলো বাইরে ঘুরে আসি	It depends on the task. If the dataset contains mixed English and Bengali in the same sentence (code-mixing), such as "The weather is so beautiful today, চলো বাইরে ঘুরে আসি", a decoder-only Transformer (e.g., GPT-style) is generally a better choice because it models multilingual text as a single sequence and can naturally continue or generate mixed-language text. If the task is translation (English <-> Bengali), then an encoder–decoder Transformer is the preferred architecture, since it is specifically designed to map one language to another.
I have a real world problem. I need to build RAG system on document corpus. Should I be using encoder-decoder model or just decoder is sufficient?	Architecturally speaking, an Encoder-Decoder model (like T5 or BART) is the theoretical ideal for RAG. However, in the real world today almost everyone builds RAG using Decoder-only models (like GPT, Claude, or LLaMA) Because modern Decoders have massive context windows and zero-shot prompting capabilities. Instead of using two separate blocks, you just paste your retrieved documents directly into the prompt along with the user's question. For your task, Use Encoder-Decoder if you are fine-tuning a small, highly specialized, low-latency pipeline. Use Decoder-Only if you want to get a powerful RAG system up and running quickly using standard API prompting!
What's Retrieval -Augmented QA task ?	Retrieval-Augmented Question Answering (Retrieval-Augmented QA) is a task where the model first retrieves relevant documents or passages from an external knowledge source, and then uses that retrieved information to generate an answer. This allows the model to answer questions using up-to-date or domain-specific knowledge instead of relying only on what it learned during training.
How does machine translation works for Decoder only transformer model since this model is trained for next token prediction? Do we need to do instruction based fine tuning with model taking a role of translater?	A decoder-only Transformer can perform machine translation by treating it as a text generation task. Given a prompt like "Translate to Bengali: The weather is beautiful today", it predicts the translated sentence one token at a time. To do this well, the model is usually instruction-tuned or fine-tuned on translation examples, although very large pretrained models can often perform translation reasonably well through prompting alone.

Any Example of pretrained Model for Document Intelleigence? Which is best and best in Market.	If you are building this in-house and want an open-source model, The gold standard is the LayoutLM or Donut. These are specialized Encoder models. Unlike standard BERT, they are trained to understand the visual layout (bounding boxes) of a page, making them perfect for extracting data from invoices, receipts, and forms. If you are working at a large company, you usually don't build this from scratch. You use dedicated Document AI APIs that handle the OCR and extraction pipelines for you, such as Google Cloud DocumentAI, Azure Document Intelligence, or AWS Textract.
What is the internal working ok CLM(Causal Language Model). Can it be explained in details?	A Causal Language Model (CLM) is trained to predict the next token using only the tokens that come before it. During training, the input passes through masked self-attention, where each token can attend only to itself and previous tokens. This produces contextual representations, which are then processed by the feed-forward layers. Finally, a linear layer and softmax generate a probability distribution over the vocabulary, and the model learns by comparing the predicted next token with the actual next token using cross-entropy loss. During inference, the model starts with the given prompt, predicts the next token, appends it to the input, and repeats this process autoregressively until the complete text is generated. The causal mask ensures that the model never uses future tokens while making a prediction, making training consistent with how generation works.
In a decoder-only model, since there is no encoder, where do the Key (K), and Value (V) vectors come from, and how does the model perform attention without encoder outputs?	In decoder-only models, Q, K, V all come from the same sequence (prompt + generated tokens)
What I understood from 'Why Decoder-Only Became the Industry Standard' — are encoders, and encoder-decoder architectures no longer useful? If yes, what are the average success rates/performance percentages for common use cases?	Hahaha, They are definitely NOT dead! Decoder-only models just became the industry standard specifically for general-purpose, conversational chat (which is what the market wants most). However, if your task doesn't require generating new text, using a massive Decoder is a waste of compute. Encoders (BERT) are still the undisputed kings of classification, spam detection, search algorithms, and embeddings. Encoder-Decoders (T5) are still heavily used for dedicated, high-volume translation and summarization pipelines where you have a fixed input text. Let me give you an example from the industry. We don't just look at raw accuracy, we look at efficiency, again i say EFFIEIENCY AND COST EFFECTIVENESS. For a task like categorizing emails, a fine-tuned BERT (Encoder) model might hit 95% accuracy in just 10 milliseconds for pennies. A massive Decoder model might hit 96% accuracy, but take 100x longer to run and cost 100x more.
if Encoder has limited generation ability why BERT is still using it?	BERT is not designed for text generation. Its goal is to understand text, not generate it. An encoder can see the entire input sentence, making it excellent for tasks like classification, sentiment analysis, question answering, and named entity recognition. For these understanding tasks, generation is unnecessary, so an encoder-only architecture like BERT is a better choice.

Compute wise is there any difference between a decoder only model and an encoder-decoder model? As per industry standards if we are choosing decoder only model does it have any compute edge?	Yes. Decoder-only models are generally simpler and more compute-efficient to train and deploy because they use a single stack of Transformer blocks instead of separate encoder and decoder stacks. This simplicity is one reason why most modern LLMs (e.g., GPT, Llama) are decoder-only. However, for tasks like machine translation or summarization, encoder–decoder models can often achieve better quality for the same model size because the encoder specializes in understanding the input while the decoder specializes in generation. So the choice depends on the application: decoder-only models are preferred for general-purpose LLMs, while encoder–decoder models remain strong for sequence-to-sequence tasks.
So ... if I have a single data source (does stationarity of data matter here ???) and I need to classify and categorize data then encoder is the choice by default ?	Yes if you want to classify data for that you need to understand the whole context, and encoder only models are good at it
How the RAG architecture works with Encoder Only Models? whenthere is no Encoding layer	RAG does not require an encoder-only model. In the original RAG architecture, an encoder is used for retrieval (to encode the query and retrieve relevant documents), while a generator (typically an encoder–decoder model like T5 or BART) generates the final answer. If an encoder-only model such as BERT is used, it is generally only for the retrieval or reranking stage, not for text generation. Since BERT cannot generate text, it must be paired with a separate generative model to produce the final answer.
Suppose we are builing subjective Q&A machine instead of objective Q&A .. which model would you recommend? - Encoder only, Decoder only or Encoder - Decoder only?	decoder-only is the best choice for subjective Q&A
fine-tune GPT-2(decoder style) to perform encoder-decoder style tasks is possible?	Yes it is possible, but then you need to format the training data accordingly. Like you can concate input and output texts and train decoder only model as they are single sequence
reiterating my question How the RAG architecture works with Decoder Only Models? like GPT models	i think prof addressed this also ... In decoder-only RAG, there is no internal encoder. Instead, a separate retriever finds relevant documents, and they are appended to the prompt. The decoder-only model then treats everything (question + retrieved text) as one sequence and generates the answer using next-token prediction.
we have microsoft copilot in our organistion, and if I turn on my deep thinking model GPT5.1 , it read everything as in all the documentation, and answers all teh questions so well, do you think encoder decoder will have a good accuracy	Your Copilot is a massive Decoder-only model. It achieves that incredible accuracy through sheer scale and zero-shot prompting! Even though it doesn't have an Encoder, its context window is so massive that it can digest all your documentation in the prompt and use its internal self-attention to connect all the dots perfectly. Would an Encoder-Decoder be accurate? The ANSWER IS YES YES YES! In fact, architecturally, Encoder-Decoders are specifically designed to read source documents and generate answers. If you took a smaller Encoder-Decoder (like FLAN-T5) and fine-tuned it on your company data, it would be highly accurate and drastically cheaper to run than Copilot. BUT BUT, who is going to do all of that training and finetuning, its very tedious process. The industry standard became Decoder-only because models like GPT don't require any fine-tuning, you just prompt them, and they can handle open-ended chat perfectly.
In an encoder decoder Transformer cross attention in each decoder layer takes three inputs: Q, K, and V. Where does each come from encoder or decoder, why is it so?	In encoder decoder transformer, in the decoder stage there are two things one is self-attention and other is cross-attention. For Self attention Q,K,V comes from decoder, and for cross-attention K and V comes from encoder and Q comes from decoder
At pricing level, will there be any difference between encoder-decoder and decoder only models?	Usually encoder-decoder models are cheaper.

How do we fine tune pretrained models? Through prompt only?	Good question. A lot of people mix these up, but Prompting is NOT Fine-Tuning! Prompting which is In-Context Learning, You are just giving the model temporary instructions. The model's internal math/weights do not change. If you clear the chat, it forgets everything. FOR Fine-Tuning, You are running an actual training loop to permanently change the math (the neural network weights) inside the model's "brain" so it learns a new skill or tone. How do we fintune?, will come up in future sessions as well, Instead of retraining the whole massive model (which costs millions), the industry standard today is PEFT (Parameter-Efficient Fine-Tuning), specifically a method called LoRA. We "freeze" the massive base model and just attach a tiny, trainable mathematical "adapter" to it.
could you give me the reference to the research paper which descirbes this pls.	its the attention is all you need paper by Vaswani et al 2017
My question about can be re-architected to encoder-decoder style?	is this your last question, fine-tune GPT-2(decoder style) to perform encoder-decoder style tasks is possible? Yes it is possible, but then you need to format the training data accordingly. Like you can concate input and output texts and train decoder only model as they are single sequence
Can I handover the output of a specialized lightweight ml model a broadbased gpt (encoder/decoder) ?	Sure, Handing off data from a specialized ML model to a massive LLM is an industry best practice! It's called a Hybrid ML Pipeline. You use the small, lightweight model to do the rigid, mathematical, or classification heavy-lifting (because it is fast, cheap, and doesn't hallucinate). Then, you pass its output (usually as a JSON string) directly into the prompt of the GPT model so it can do the flexible, human-like reasoning and summarization. But if your ML is not accurate then GPT will try to reason or summarize wrong output. If your specialized ML model makes a mistake, the GPT model will blindly trust it. The LLM will use perfect logic and beautiful language to reason over completely incorrect data! Your final output is entirely dependent on the accuracy of your first, lightweight model.
Transformer models also need supervised learning as we are using input and target ?	Yes when you want to finetune a pre-trained model on your task-specific data, you need to train it using the input output pair
is there any Dataset build by IIT KGP ?	Yes there are datasets like HateXplain, IndoRE, You can find some of the datasets from https://cnerg-iitkgp.github.io/datasets.html

- As an individual I have no real hope of building a transformer model or building one and training one. - I can use a transformer but never build one unless it is a commercially backed. - Transformers are always trained on a large corpus of data for it to be meanningful - If I will never build a transfomer model ground up then what advantage do I gain from learning the details (... I luv learning and want to know .. but I am curious) Or - If I am wrong, I should be able to build a small transformer and train it on a small corpus of data, just like a Decision Tree or LogisticRegression Model Above are some of my confusing thoughts .. I wonder which ones are correct and which ones are wrong	You absolutely can build a small Transformer from scratch (in a few hundred lines of Python) and train it on a small dataset using your laptop or a free Google Colab notebook. Why learn the details if you just use APIs? Because it gives you a massive engineering advantage! Understanding the architecture helps you realize why long prompts cost so much money, how to debug hallucination errors, and how to use cheap fine-tuning tricks (like LoRA) to customize commercial models without paying a fortune! Search YouTube for "Let's build GPT: from scratch, in code, spelled out" by Andrej Karpathy. He literally builds a working Transformer from an empty file and trains it on a tiny dataset right before your eyes. You can absolutely do this! https://youtu.be/kCc8FmEb1nY
is the len of token fixed? example if there a posibility to tokenize lgo, toschool?	Can you please elaborate what do you mean by fixed token? Basically token generated depends on model tokenizer

diff between seqential and functional module from torch	sequential - > it chains layers in a fixed, linear order. Input passes through each layer one by one, automatically.
	import torch.nn as nn
	
	model = nn.Sequential(
	nn.Linear(784, 256),
	nn.ReLU(),
	nn.Linear(256, 128),
	nn.ReLU(),
	nn.Linear(128, 10)
	)
	
	# Forward pass is automatic — no need to write forward()
	output = model(input)
	
	functinal --> base class you subclass to define any architecture you want. You manually write the forward() method, giving you full control.
	
	import torch.nn as nn
	import torch.nn.functional as F
	
	class MyModel(nn.Module):
	def __init__(self):
	super().__init__()
	self.fc1 = nn.Linear(784, 256)
	self.fc2 = nn.Linear(256, 128)
	self.fc3 = nn.Linear(128, 10)
	self.dropout = nn.Dropout(0.3)
	
	def forward(self, x):
	x = F.relu(self.fc1(x))
	x = self.dropout(x)
	x = F.relu(self.fc2(x))
	return self.fc3(x)
	
	model = MyModel()
	output = model(input)

You mentioned we are loading only 5,000 pairs from Samantar due to Colab limitations. How much does translation quality drop compared to training on the full 11.8 million pairs? Is it quality issue or quantity issue?	Most of the time, training quality depends on the amount of data, given the data quality is good. So if I assume training pairs are good quality then performance will be poor when training with 5000 pair compared to training with 11.8M pairs
While loading the dataset how to figure out which paratmeters are mandatory and which are not? How its will match with the provided arguments?	The rule is to look for the Equals Sign (=) in the documentation. Mandatory: Parameters with NO equals sign (e.g., dataset). If you don't provide these, Python will crash with a TypeError. Optional: Parameters WITH an equals sign (e.g., batch_size=32). These have "default" values. If you skip them, Python just uses the default!

How to stram for two languages like hindi and kannada?	In this dataset showed in the example, it is from English to other Indian languages. But there are other datasets available from an Indic language to other like BPCC
def load_dataset(path: str, name: Optional[str] = None, data_dir: Optional[str] = None, data_files: Optional[Union[str, Sequence[str], Mapping[str, Union[str, Sequence[str]]]]] = None, split: Optional[Union[str, Split, list[str], list[Split]]] = None, cache_dir: Optional[str] = None, features: Optional[Features] = None, download_config: Optional[DownloadConfig] = None, download_mode: Optional[Union[DownloadMode, str]] = None, verification_mode: Optional[Union[VerificationMode, str]] = None, keep_in_memory: Optional[bool] = None, save_infos: bool = False, revision: Optional[Union[str, Version]] = None, token: Optional[Union[bool, str]] = None, streaming: bool = False, num_proc: Optional[int] = None, storage_options: Optional[dict] = None, **config_kwargs,) how “hi” = Hindi is mapped here?	You are right, "language" or "hi" is not explicitly defined in the signature. Most of the time in Hugging Face, it is mapped to the name parameter! The path is the main dataset (like "samanantar"), and the name is used as the "subset" (like "hi" for the Hindi slice of the data). Also, Look at the very last parameter in your list **config_kwargs. The double asterisk (**) in Python means "Catch-All". If you pass a parameter that isn't explicitly listed, Python won't crash. It gathers those extra parameters, puts them in a dictionary, and passes them through config_kwargs so the specific dataset script can use them.
if google translator is not a model then what it is?	Google translator is a model trained for translation
Here can we change target language? To any other say bn?	You can take EN-BN pair for it
did not understand the ground truth	example: suppose you are doing MT with pair En: xxxx, Bn: yyy and model generates BN: zzz, then yyy is the ground truth
what is ground truth?	example: suppose you are doing MT with pair En: xxxx, Bn: yyy and model generates BN: zzz, then yyy is the ground truth

ds = load_dataset(
"ai4bharat/samanantar",
"hi", # "hi" = Hindi
split="train",
streaming=True, # stream so we only download what we need
trust_remote_code=True
)
 	
in this how to pass parameter for two languages like [“hi”, ”ka”]
	It is super logical to try a list, but it will crash!
	
	The name parameter strictly requires a single string (like opening one drawer at a time). If you want multiple languages, you have to load them as separate datasets first!
	
	Example code to zip them, try something like this -
	
	from datasets import load_dataset, interleave_datasets
	
	ds_hi = load_dataset("ai4bharat/samanantar", "hi", split="train", streaming=True)
	ds_ka = load_dataset("ai4bharat/samanantar", "ka", split="train", streaming=True)
	
	ds_multilingual = interleave_datasets([ds_hi, ds_ka])

Is Ground Truth the one one we translated using the model or is it a default set value ?	example: suppose you are doing MT with pair En: xxxx, Bn: yyy and model generates BN: zzz, then yyy is the ground truth
i got this error: RuntimeError: Internal: src/trainer_interface.cc(584) [(static_cast<int>(required_chars_.size() + meta_pieces_.size())) <= (trainer_spec_.vocab_size())] Vocabulary size is smaller than required_chars. 30 vs 171. Increase vocab_size or decrease character_coverage with --character_coverage option.	This is a classic tokenizer error! Before SentencePiece learns words, it must add every single unique letter/symbol from your dataset into its vocabulary so it can spell anything. It found 171 unique base characters in your text, but you set your total vocab_size to only 30. It physically cannot fit the alphabet into a box of 30 slots! You need to increase your vocab_size! For Machine Translation, a vocabulary of 30 is way too small (you might have missed some zeros). Try changing it to vocab_size=8000 or whatever size you think is appropriate. (in the notebook its set to 4000)
Do we have to use BOS_ID = 1 and EOS_ID = 2? Please elaborate	It is upto you but whatever you choose make it consistant through out the training. But this are default setting, no need to change
i was confused between the googleTranslate version and the Ground Truth version.. Could you pls tell me the difference between them ?	Suppose you translate a Hindi sentence to English using google translate. Then is is google translated version. And if you know the correct English translation that is ground truth. Basically ground truth is anything you use to teach your model what is correct and what is wrong. If you feel google version is good you can use it as ground truth
where is the vocabulary has imported?	You actually won't find an import vocabulary statement in the code! That is because the vocabulary doesn't exist yet. The tokenizer in this notebook builds the vocabulary completely from scratch by reading your specific dataset! When you run the SentencePieceTrainer.train(...) function, it scans your data, figures out the best sub-words, and saves them to your hard drive as a .model and a .vocab file. Later in the code, you simply load that .model file to give your network its vocabulary. Just an Analogy, We aren't buying a pre-written dictionary from a bookstore (importing). We are forcing the computer to read our specific Hindi-English dataset and write its own custom dictionary based only on the words it sees!
When doing translation, what is the role of padding exactly?	Model can process a fixed size context, so if the sentences are different size to make them same size we use padding
is vocab size always same between translation language? what are the scenarios we have it different or not equal	Nope! The Encoder and Decoder use two completely separate embedding tables. You can easily set src_vocab_size = 10000 and tgt_vocab_size = 8000 without any math errors. English only needs ~100 base characters (a-z, punctuation). Hindi (Devanagari) needs hundreds of base characters for consonants, matras, and conjuncts. The Hindi tokenizer needs a larger vocabulary just to cover its alphabets.
Can we split train test data into other percent like 70-20	Yes but make sure you have enough data to train the model
Why did we tokenize before train test split and only on 4000 pairs when we have 5000 pairs in the data?	Basically we want to learn tokenizer from training data
my train the model section is giving error: The size of tensor a (281) must match the size of tensor b (200) at non-singleton dimension 1	Same error you got 113 , 100 error you asked previously, try to keep on increasing it and see what happens or try the chopping of extra words
When we are using libraries like HuggingFace- we are using predefined datasets for translation but it should be using the models to translate and generate output- does it not use tokens when generating? basically question is whether translation is realtime	In realtime you use already trained models, just inference happening
Will the size of num_encoder_layers and num_decoder_layers should always be same?	No, they do not need to be the same.
Can the number of encoder layer and decoder layer can be different..??, if yes then what impact it will have on the training and inference	Yes they can be different. more layer can help better learning, but inference can be costly. Also more layer can lead to model overfit

What are actually encoder decoder layers? does it run in loop?	You pass a sentence through encoder layers(there are multiple encoder block), then you get final representation, then decoder start generating in loop(number of tokens you want to generate)
Follow Up: In realtime you use already trained models, just inference happening Query: Who pays for token as calls are not free?	If you use paid MT then you are paying, but suppose you are using google translate then google is paying for it. Nothing is free someone has to pay for the infrastructure
RuntimeError: The size of tensor a (113) must match the size of tensor b (100) at non-singleton dimension 1	PyTorch is telling you that you are trying to combine a matrix of size 113 with a matrix of size 100. In Transformers, "Dimension 1" is your Sequence Length. 113 = A really long Hindi/English sentence your DataLoader just pulled. 100 = The hardcoded max_len of your model (usually inside your Positional Encoding layer). You are trying to fit a 113-word sentence into a 100-word box FIX WOULD BE Increase the limit, Find where you defined your model or Positional Encoding and change max_len=100 to max_len=200 or higher to accommodate longer sentences. OR Truncate the text, In your data pipeline, add code to slice off anything past 100 tokens: tokens = tokens[:100].
backpropogation it self goes back to trace and improve whatn epoch is same or its saperate step after back propogation ?	Backpropagation is an action (the math that updates the weights). An Epoch is a measurement of data (going through the entire dataset one time). An epoch is not a step after backprop; instead, one single epoch contains many backpropagations!
The models those are trained in code or notebooks or anywhere- suppose the prams are in billions to there is a huge cost and compute that was used to train. where does these models get saved or should be saved so training isnt lost.	You can save them locally and use later or upload to platform like huggingface where others can also download your model and use
where the model is saved ?	if you don't provide any specific path, then it will save to your current working directory
backpropogation vs epoch	Backpropagation is the mathematical process of calculating errors and updating the neural network's weights. This happens every single Batch. Epoch is like a milestone marker. It simply means the model has looked at the entire dataset exactly one time.
where is it saved - "simple_model.pt "	if you don't provide any specific path, then it will save to your current working directory
Can we train the model again using the saved model file ?	Yes you can reload the model weights from the file and continue training
what is the use of register_buffer ?	register_buffer is a special way to save a tensor inside your model. It tells PyTorch "This is NOT a learnable weight, do not update it during backprop. BUT, it is an official part of the model, so make sure you move it to the GPU and save it in the state_dict. You are likely seeing this in the Positional Encoding layer. That matrix of sines and cosines is mathematically fixed. We don't want the optimizer to change it. But we do need it to live on the exact same GPU as our input data, so we "register" it as a buffer.

So as per diagram, we have 1 encoding and decoding layer. but in practice llms have how many blocks?	Yes inside a encoding or decoding block, there will be multiple encoding and decoding layer stacked one upon another